

TQ1-IR: A Compiler Dialect for Ternary Weight Neural Networks

Nuno Miguel Pedrosa Pombo

Abstract

Running large language models on consumer hardware is constrained by memory bandwidth, not compute. Ternary weight networks like BitNet b1.58 address this directly: weights in $\{-1, 0, +1\}$ halve memory traffic compared to 8-bit quantization and open the door to multiplier-free systolic arrays. The tooling, however, has not kept up. There is no open-source compiler pipeline that takes a ternary model from its on-disk format through a typed intermediate representation to a hardware instruction set. This paper describes TQ1-IR, a Python-based IR dialect inspired by MLIR, with a GGUF frontend, three optimization passes, and a validation backend. Applied to BitNet b1.58 2B4T, the compiler processes 332 tensors into 30 layer functions, fuses 150 RMSNorm-Linear pairs eliminating 600 KB of intermediate buffers per layer, and produces an instruction trace where TGEMV operations account for 50% of all instructions. Weight decoding matches the reference engine exactly, with a maximum absolute difference of zero across all tested tensors.

1. Introduction

The dominant cost of transformer inference is not arithmetic. It is moving weights from memory to the compute units. A 7-billion-parameter model at float16 occupies roughly 14 GB; reading those weights once per generated token at the memory bandwidth of a typical CPU takes several hundred milliseconds. This is the Memory Wall, and it has driven nearly all recent work on model compression.

Ternary weight quantization is one of the more aggressive responses. BitNet b1.58 [MA24] trains models from scratch with weights constrained to $\{-1, 0, +1\}$. The per-weight storage cost drops to roughly 1.58 bits, a factor of ten compared to float16. More importantly, the arithmetic changes character: a matrix-vector product with ternary weights requires only additions and subtractions, no multiplications. This makes the operation well-suited to bit-serial systolic arrays, a class of hardware accelerator that has been studied for decades but rarely had a production workload to justify the silicon.

The model side of this story is largely solved. BitNet b1.58 2B4T runs correctly on commodity CPUs, and the bitnet.cpp project provides a working inference engine. The compiler side is not solved. There is no open compiler pipeline that:

1. accepts a ternary model in GGUF format as input,
2. represents its computation in a typed intermediate representation,
3. applies optimization passes that are aware of the ternary structure, and
4. emits instructions for a target accelerator ISA.

This paper describes TQ1-IR, which fills that gap. The implementation is in Python and deliberately does not depend on the MLIR C++ infrastructure, making it runnable on a consumer laptop with no GPU. We validate the compiler by comparing its weight decoding against the reference engine on the actual BitNet b1.58 2B4T model weights.

The contributions are:

- A typed SSA intermediate representation for ternary transformer inference, with GGUF frontend and symbolic ISA backend.
- Three compiler passes: type verification, norm-linear fusion, and activation quantization lowering.
- A documented correction to the T36 weight format, which is misdescribed in at least one reference implementation.

2. Background

2.1 BitNet b1.58 and the T36 Format

BitNet b1.58 [MA24] is a transformer architecture trained end-to-end with ternary weights. The model used in this work, BitNet b1.58 2B4T, has 30 transformer layers, a model dimension of 2560, 20 attention heads with 5 key-value heads (grouped-query attention), a feed-forward dimension of 6912, and a vocabulary of 128,256 tokens. The total parameter count is approximately 2.4 billion.

Weights are stored in GGUF files using GGML type 36, referred to here as T36. The on-disk format packs four weights per byte: byte p of a block encodes the weights at positions p , $p+32$, $p+64$, and $p+96$ using 2-bit codes in bit pairs (7:6), (5:4), (3:2), (1:0). The code-to-weight mapping is 0 to -1, 1 to 0, 2 to +1.

Determining the correct block layout required reading the actual tensor offsets from the GGUF file. The format is: all packed weight bytes for the tensor ($n/4$ bytes, where n is the number of weights), followed by 32 bytes of trailing data, of which the first 4 bytes are a float32 global scale for the entire tensor. This is not interleaved. At least one other implementation describes the format as 34 bytes per block of 128 weights (32 bytes packed, 2 bytes float16 scale), which would also give the correct total file size but produces incorrect results when the scale bytes are read: they contain packed weight data from adjacent blocks, and some of those bytes encode float16 NaN values.

2.2 MLIR and Why We Did Not Use It

MLIR [LA21] is a compiler infrastructure that allows custom dialects to be defined, composed, and lowered through a sequence of passes. It is the obvious framework for this kind of work. We chose not to use it for one reason: the build toolchain requires several gigabytes of dependencies and a C++ compiler, which makes the barrier to entry high enough that the compiler would not run on the hardware the model is designed for.

TQ1-IR is structurally identical to an MLIR dialect. It uses static single assignment form, typed values, operation classes with operand and result lists, and a textual IR format that could be parsed by an MLIR tool with a compatible grammar. The implementation is pure Python. A port to actual MLIR would preserve the design and require only replacing the data structures.

3. The TQ1-IR Dialect

3.1 Type System

TQ1-IR has three tensor types. TernaryTensorType represents weights in $\{-1, 0, +1\}$ with an associated group size (128 for T36). F32TensorType and I8TensorType represent activations at float32 and int8 precision. The shape field uses -1 for dynamic dimensions (sequence length).

In the textual IR, these print as:

```
!tq1.ternary<2560x2560, g128>
!tq1.f32<?x2560>
```

```
!tq1.i8<?x2560>
```

3.2 Operations

The base dialect has eight operations:

- **EmbedOp** -- token embedding lookup.
- **TernaryLinearOp** -- matrix-vector product with ternary weight matrix. This is the TGEMV instruction, the primary operation of the systolic array.
- **RMSNormOp** -- root-mean-square layer normalization.
- **RoPEOp** -- rotary positional encoding (NEOX-style, theta=500000).
- **AttentionOp** -- grouped-query attention with causal mask.
- **SiLUMulOp** -- SiLU activation applied to the gate projection, multiplied element-wise with the up projection.
- **ReturnOp** -- function return.

3.3 Module Structure

A Module contains a weight table and a list of functions. The weight table stores WeightEntry records: name, type, byte offset in the GGUF file, byte size, and GGML type code. Weights are lazy references. No weight data is loaded until the kernel backend requests it.

Each transformer layer is compiled to a Function named layer_N. The function takes two arguments: the activation tensor (%xN, shape ?x2560) and position indices (%posN, shape ?). The body is a Block of SSA operations. A representative layer function, before optimization passes, is shown below:

```
tq1.func @layer_0(%x0: !tq1.f32<?x2560>, %pos0: !tq1.f32<?>)
-> (!tq1.f32<?x2560>) {
  %attn_norm_0 = tq1.rms_norm %x0, @blk.0.attn_norm.weight {eps=1e-05}
  %q_0 = tq1.ternary_linear %attn_norm_0, @blk.0.attn_q.weight
  %k_0 = tq1.ternary_linear %attn_norm_0, @blk.0.attn_k.weight
  %v_0 = tq1.ternary_linear %attn_norm_0, @blk.0.attn_v.weight
  %q_rope_0 = tq1.rope %q_0, %pos0 {theta=500000.0}
  %k_rope_0 = tq1.rope %k_0, %pos0 {theta=500000.0}
  %attn_out_0 = tq1.attention %q_rope_0, %k_rope_0, %v_0
    {n_heads=20, n_kv=5, hd=128}
  %o_proj_0 = tq1.ternary_linear %attn_out_0, @blk.0.attn_output.weight
  %ffn_norm_0 = tq1.rms_norm %o_proj_0, @blk.0.ffn_norm.weight {eps=1e-05}
  %gate_0 = tq1.ternary_linear %ffn_norm_0, @blk.0.ffn_gate.weight
  %up_0 = tq1.ternary_linear %ffn_norm_0, @blk.0.ffn_up.weight
  %act_0 = tq1.silu_mul %gate_0, %up_0
  %out_0 = tq1.ternary_linear %act_0, @blk.0.ffn_down.weight
  tq1.return %out_0
}
```

The IR does not currently represent residual additions. These are additions of two float32 tensors, which do not involve the ternary hardware and will be handled by the scalar unit. They will be added in a future revision.

4. Compilation Passes

4.1 Type Verification

The verify pass checks the IR for type errors before any transformation runs. It validates that all weight references resolve to entries in the module weight table, that activation types are consistent across operations (RMSNorm and RoPE require F32 input, TernaryLinear accepts F32 or I8), that matrix dimensions are compatible (allowing -1 wildcards for dynamic shapes), and that GQA parameters satisfy $n_heads \% n_kv_heads == 0$.

Applied to the BitNet b1.58 2B4T model, the pass reports 0 errors and 0 warnings across all 420 operations in 30 functions.

4.2 Norm-Linear Fusion

BitNet layers apply RMSNorm immediately before every linear projection. In the unfused IR, each norm operation writes a full activation tensor to memory before the linear reads it back. The fusion pass identifies RMSNorm-TernaryLinear pairs where the norm output has no other consumers and replaces them with a single FusedNormLinearOp that computes the norm and the projection without materializing the intermediate buffer.

The pass handles the attention sublayer correctly. The attn_norm output feeds three projections (Q, K, V), so the original RMSNorm has three consumers. The fusion replaces it with three FusedNormLinearOps, each carrying a reference to the norm weight. The intermediate buffer is still eliminated; the norm is recomputed three times. For the hardware target, this is preferable: on a memory-bandwidth-limited system, recomputing 2560 multiplications costs less than reading and writing 10 KB.

Results for the full model: 60 RMSNorm operations absorbed into 150 FusedNormLinear operations. 60 intermediate buffers eliminated. At a model dimension of 2560 floats at 4 bytes each, this is 10,240 bytes per elimination, or 600 KB per layer.

4.3 Activation Quantization Lowering

The quantization lowering pass makes the precision of activations explicit in the IR. It inserts a QuantizeOp (F32 to I8 via per-row absmax scaling) after each TernaryLinear or FusedNormLinear, and a DequantizeOp (I8 to F32) before any operation that requires float32 input (RMSNorm, RoPE).

This pass transforms the implicit float32 data flow into an explicit mixed-precision graph. The hardware backend can use this information to route activations through integer datapaths between ternary operations, loading float32 only when needed.

Applied to the model after fusion: 210 quantization operations inserted (one Quantize after each of the 150 FusedNormLinear ops and 60 remaining TernaryLinear ops, plus paired Dequantize ops before norm and rope operations that receive quantized values).

5. Backend

5.1 ISA Emitter

The ISA backend translates TQ1-IR to a symbolic assembly language for the target accelerator. The instruction set has eight instructions: EMBED, TGEMV, TGEMV_N (fused norm and TGEMV), RMSNORM, ROPE, ATTN, SILU_MUL, and RET. QUANT_I8 and DEQUANT instructions are added by the quantization lowering pass.

For the unfused IR, TGEMV accounts for 210 of 420 total instructions, or exactly 50%. After fusion, the same computation is expressed with FusedNormLinear (TGEMV_N) replacing the separated RMSNORM-TGEMV pairs. The systolic array is involved in every second instruction in the schedule.

The weight table summary produced by the ISA emitter reports 210 ternary tensors (497 MB) and 122 dense tensors. The ternary tensors are the ones that stream through the systolic array; the dense tensors (norms, embeddings, sub-norm weights) go through conventional vector units.

5.2 Kernel Backend

The kernel backend is a NumPy executor that runs the IR directly. Its purpose is validation: if the kernel backend produces the same outputs as the reference engine for the same weights and inputs, the frontend and passes are correct.

The backend uses lazy mmap-based weight loading. Weights are decoded from T36 format to float32 on demand and cached. The T36 decoder, after the format correction described in Section 2.1, reads $n/4$ bytes of packed weights followed by a float32 global scale. The int8 activation path (via QuantizeOp and DequantizeOp) is implemented but uses a simplified dequantization (without stored per-token scales) that is sufficient for data-flow validation.

Layer execution is fully traced. Each operation produces a result tensor, and the backend verifies after each step that the output is finite and non-constant before proceeding.

6. Evaluation

6.1 Model Coverage

The GGUF frontend processes BitNet b1.58 2B4T in 0.43 seconds on a consumer laptop (Intel Core i3-1115G4, no GPU). It registers 332 tensors and constructs 30 layer functions. Architecture metadata is read from the GGUF key-value section using automatic prefix detection (the model uses the "bitnet-b1.58." prefix rather than the standard "llama." prefix).

6.2 Pass Results

Table 1. Summary of pass results on BitNet b1.58 2B4T.

Verify pass: 0 errors, 0 warnings.

Fusion pass: 60 RMSNorm ops absorbed, 150 FusedNormLinear ops created, 60 intermediate buffers eliminated (600 KB/layer).

Quantization lowering: 210 quantization operations inserted.

ISA instruction count (unfused): 420 total; TGEMV 210 (50%), RMSNORM 60 (14%), ROPE 60 (14%), ATTN 30 (7%), SILU_MUL 30 (7%).

6.3 Weight Decoding Accuracy

Four weight tensors were decoded by TQ1-IR and compared against the output of the reference engine (mom/engine, a separately implemented GGUF reader for the same model). All four report max absolute difference of 0.00 at float32 precision:

- blk.0.attn_q.weight (2560x2560 ternary)
- blk.0.attn_v.weight (640x2560 ternary)
- blk.0.ffn_gate.weight (6912x2560 ternary)

- blk.5.attn_q.weight (2560x2560 ternary, layer 5)

The kernel backend executes layer 0 on a random input (4 tokens, float32) in 0.48 seconds. All 13 operations produce finite, non-constant outputs.

7. Future Work

The most immediate gap is residual additions. The current IR models the computation inside each sublayer but does not represent the skip connections that add the layer input back to the sublayer output. Adding a `ResidualAddOp` is straightforward; the operation has no hardware novelty and the main question is where in the pass order it interacts with quantization lowering.

The ISA definition is a natural fit for TAIDL [AC25], which takes an accelerator ISA description and generates a compiler backend automatically. Mapping TQ1-IR to a TAIDL specification would produce a backend that targets real FPGA or ASIC implementations rather than symbolic assembly.

On the model side, the evaluation here is limited to BitNet b1.58 2B4T. Falcon-E-3B [FE25] is a ternary model with 32K native context trained at 1 billion and 3 billion parameters. Adapting the frontend to its architecture (which omits the BitNet sub-norm layers present in the 2B4T model) would broaden the compiler coverage without requiring changes to the pass infrastructure.

Whoever implements the TQ1-IR ISA on actual FPGA or ASIC hardware is requested to contact the author.

8. Conclusion

TQ1-IR is a working compiler pipeline for ternary weight transformers. It reads a production model from disk, constructs a typed SSA representation, applies three optimization passes, emits symbolic hardware instructions, and validates weight decoding against an independent reference. The implementation requires no external compiler infrastructure and runs on the same consumer hardware the model targets.

The key finding from the implementation work is practical: the T36 weight format is not interleaved 34-byte blocks, it is a packed section followed by a single float32 scale, and getting this wrong produces NaN values that are not immediately traceable to the format misreading. Getting it right requires reading the actual tensor offsets from the GGUF file and checking the gaps.

References

- [MA24] Ma, S. et al. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764, 2024.
- [LA21] Lattner, C. et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. CGO 2021.
- [AC25] ACT: Auto Compiler Tool for Tensor Accelerators. tiuae/ACT, 2025.
- [FE25] Falcon-E: Ternary Large Language Models. Technology Innovation Institute, 2025.